---

# Claude Code Prompt

You are a principal engineer, quantitative researcher, prediction-market microstructure analyst, and trading-systems architect.

Your task is to design the **actual MVP codebase and implementation blueprint** for a **Polymarket trader-intelligence and alpha-detection system**.

This system must pursue **two related but distinct targets**:

1. **Near-term repricing edge**
    - detect whether skilled trader flow predicts future price movement after the observed trade
2. **Direct final-resolution edge**
    - detect whether trader-conditioned signals improve prediction of final market resolution beyond market price alone

This is important:
Do **not** reduce the system to “copy high win-rate traders.”
That is too weak and structurally confounded.

Instead, build a system that treats this as a **trader-classification + signal-extraction + prediction problem**.

The system must infer whether some observed trader behavior contains:

- **price-relative edge**
- **final-outcome edge**
- **topic-specific edge**
- **copyable edge after realistic frictions**

The system must also explicitly separate:

- whales
- serious non-whales
- market-makers
- directional discretionary traders
- hedgers / arbers where possible
- copiers / shadow wallets
- noise traders

## Hard framing requirement

The system must support **both** of these research theses simultaneously:

### Thesis A: Microstructure / repricing thesis

Skill is best detected as entering before repricing.
This is measured using CLV-style or entry-vs-later-price metrics.

### Thesis B: Final-resolution thesis

Some traders may contain real informational or forecasting edge that improves prediction of final resolution, not just short-term drift.

Your design must support both.
Do not force the project into only one frame.

The docs suggest that repricing is the more plausible first signal, but they also explicitly support testing whether trader-flow features improve final-resolution prediction over market price alone. Design the MVP accordingly.

---

# Ground truth assumptions you must respect

## Data reality

Assume the first-party public data surfaces are sufficient for an MVP:

- leaderboard
- trades
- positions
- closed positions
- total position value
- market metadata and tags
- prices history
- orderbook / best bid-ask / market channel
- official real-time data stream
- CLOB clients and official CLI

These are enough to build a real system.

Assume the system can use at minimum:

- official CLOB clients in Python / TypeScript / Rust
- official `real-time-data-client`
- official `polymarket-cli`
- official Gamma/Data/CLOB surfaces
- optional official subgraph / resolution subgraph if needed later
- optional safe-wallet integration patterns if execution is enabled later

The repo triage docs indicate that this layer is already strong; the missing work is the alpha logic, not basic connectivity.

# Quant reality

Assume all of the following:

- raw win rate is not a valid primary metric
- raw PnL is not a valid primary metric
- top wallets often include market-makers and non-directional actors
- market-maker filtering is mandatory
- copier detection is mandatory
- topic specialization matters
- proxy wallet is the basic identity object for MVP
- full wallet clustering is out of MVP scope but extension points should exist
- near-term repricing and final resolution must both be tested
- all live signals must include copyability penalties
- all evaluation must be out-of-sample

# Execution reality

Execution is optional.
The MVP should default to:

- signal-only mode
- explainable ranking outputs
- paper-trading / simulation mode

Execution integration can be added later, but it must not dominate the architecture.

---

# System objective

Design a system that continuously computes:

## 1. Trader reconstruction and classification

For each wallet, estimate:

- whale status
- serious non-whale status
- market-maker likelihood
- directional trader likelihood
- copier likelihood
- noise / weak-signal likelihood

## 2. Trader feature vectors

For each wallet and topic:

- CLV-style edge
- realized edge
- final-resolution edge
- lateness-adjusted edge
- odds-difficulty-adjusted edge
- pnl margin
- directional purity
- holding-time profile
- topic-specific edge
- convergence contribution
- market-impact proxy
- copyability contribution
- uncertainty / effective sample size

## 3. Trader scores

The system must compute at least:

- global price-relative skill score
- global final-resolution skill score
- topic-specific price-relative skill score
- topic-specific final-resolution skill score
- global composite score

- topic composite score
- posterior confidence / uncertainty band
- filter flags

## 4. Market opportunity scores

The system must compute **two market scores** for each side of each current market:

### A. Repricing score

Does observed trader flow suggest future price movement?

### B. Final-resolution score

Does observed trader flow suggest the current implied probability is wrong relative to final resolution?

The system must then compute a combined:

- current opportunity score
- copyability-adjusted score
- confidence score
- veto state

---

# What the product must output

At minimum, the live product must surface:

1. ranked current market opportunities
2. which side is interesting
3. whether the signal is:
    - repricing-only
    - final-resolution-only
    - both
4. which traders contributed
5. whether they are whales or non-whales
6. whether they are topic specialists
7. whether they appear directional or MM/copier-contaminated
8. what penalties were applied
9. whether the signal still looks copyable now
10. why the signal could be wrong

This must be explainable.
Do not build a black box.

---

# Required output sections

## 1. Monorepo / repository layout

Propose a real monorepo or multi-repo layout.

Use a structure such as:

- `apps/`
- `services/`
- `packages/`
- `infra/`
- `scripts/`
- `tests/`
- `docs/`

Be explicit.

Separate clearly:

- ingestion
- normalization
- storage
- analytics
- trader classification
- scoring
- live signal generation
- backtesting
- API
- dashboard
- execution adapter
- shared schemas/types

For each folder:

- explain its role
- whether it is MVP-critical

- whether it wraps official repos or is fully custom

## 2. Recommended MVP stack

Choose one primary stack and justify it.

You may choose something like:

- Python for analytics, inference, feature engineering, backtests
- TypeScript for live stream ingestion and API/web
- Postgres for normalized serving state
- DuckDB/Parquet for research snapshots and replay
- Redis only if clearly necessary
- optional Rust only if there is a concrete reason

Explain:

- why this is the best MVP tradeoff
- which official Polymarket repos are wrapped
- where custom code starts

## 3. Exact repo/dependency selection

List the exact official and optional external repos/packages to use.

At minimum, evaluate:

- `Polymarket/py-clob-client`
- `Polymarket/clob-client`
- `Polymarket/rs-clob-client`
- `Polymarket/real-time-data-client`
- `Polymarket/polymarket-cli`
- `Polymarket/polymarket-subgraph`
- `Polymarket/resolution-subgraph`
- `Polymarket/safe-wallet-integration`
- `sculptdotfun/tremor`
- `Jon-Becker/prediction-market-analysis`
- `frazane/scoringrules`

For each one:

- role in system
- use as-is vs fork vs reference

- layer served
- whether it is MVP-critical or later-stage
- whether it is safe to build on

Use the repo-triage conclusions as prior assumptions:
official repos are strong for plumbing; custom alpha logic remains the hard part.

# 4. Canonical data model

Define the normalized internal schema.

You must include at minimum:

### Market domain

- `event`
- `market`
- `outcome`
- `market_tag`
- `market_topic`
- `market_archetype`
- `resolution_state`

### Wallet/trader domain

- `trader_wallet`
- `trader_profile`
- `trader_snapshot`
- `wallet_cluster_placeholder`
- `trader_classification`
- `trader_quality_flags`

### Trading domain

- `raw_trade_event`
- `normalized_trade`
- `raw_position_snapshot`
- `position_snapshot`
- `closed_position`
- `wallet_market_exposure`
- `wallet_market_side_exposure`

### Price/liquidity domain

- `price_snapshot`
- `prices_history_point`
- `orderbook_snapshot`
- `spread_depth_snapshot`
- `market_microstructure_snapshot`

### Feature/scoring domain

- `trade_feature_row`
- `trader_feature_row`
- `topic_feature_row`
- `trader_score_snapshot`
- `topic_score_snapshot`
- `market_signal_snapshot`
- `signal_explanation_row`
- `backtest_signal_row`

For each model specify:

- fields
- types
- PK
- indexes
- update cadence
- mutable vs append-only
- source of truth

## 5. Ingestion architecture

Design the ingestion layer in detail.

Separate paths for:

### A. Real-time ingestion

Use:

- official `real-time-data-client`
- optional market WebSocket channel
  for low-latency updates

### B. Batch polling

Use:

- leaderboard
- trades
- positions
- closed positions
- value
- market metadata

### C. Metadata sync

Use:

- Gamma events/markets/tags
- market taxonomy surfaces
- resolution metadata

### D. Historical sync

Use:

- price history
- optional subgraph / ETL
- optional archived datasets / research repos

For each path specify:

- cadence
- endpoint/source
- retry logic
- dedupe
- idempotency
- schema validation
- rate-limit handling
- failure modes

The docs emphasize that public trader-level and market-level data are directly usable, but reconstruction and identity logic remain noisy. Reflect that in the design.

## 6. Feature engineering design

Define the exact MVP features.

Split them into groups.

## A. Price-relative / CLV-style features

Examples:

- trade entry vs 1h later midpoint
- trade entry vs 6h later midpoint
- trade entry vs 24h later midpoint
- trade entry vs pre-resolution midpoint
- weighted average CLV
- median CLV
- time-to-resolution-weighted CLV
- difficulty-weighted CLV

## B. Final-resolution features

Examples:

- realized normalized value
- outcome residual vs entry price
- outcome residual vs market baseline
- topic-conditioned final hit quality
- Brier-style forecast contribution
- log-score-style contribution

## C. Behavioral / classification features

Examples:

- two-sided same-market trading ratio
- exposure churn
- mean holding period
- same-market side flipping rate
- volume / pnl margin
- size concentration
- trade burstiness
- market breadth vs depth
- lag-to-leader correlation
- overlap with suspected copier wallets

## D. Topic specialization features

Examples:

- CLV by topic
- final-resolution edge by topic
- ESS by topic
- rolling-window stability by topic
- topic concentration
- topic entropy

### E. Live market features

Examples:

- count of strong traders on side
- skill-weighted net flow
- topic-matched skilled flow
- independent convergence
- price movement since first strong trade
- spread/depth penalty
- crowding penalty
- stale signal penalty
- low-attention bonus
- time-to-resolution penalty

Define each one precisely.

## 7. Trader classification layer

Design the first-pass classification pipeline.

It must distinguish at least:

- whale
- serious non-whale
- market-maker style
- directional discretionary
- copier / shadowing wallet
- noise wallet

For each class:

- define observable heuristics
- define required features
- define likely false positives
- define what is safe for MVP
- define what should be deferred

The docs are explicit that market-maker confounding is a first-class issue. Your design must reflect that.

# 8. Trader scoring models

Design a **dual scoring framework**.

## A. Price-relative skill model

Estimate whether the trader systematically enters before repricing.

This should include:

- CLV-style score
- lateness penalty
- odds difficulty penalty
- ESS threshold
- market-maker/copier penalties
- topic-specific deviations
- uncertainty

## B. Final-resolution skill model

Estimate whether the trader improves prediction of final resolution beyond market price alone.

This should include:

- residual outcome edge
- proper scoring style contributions
- topic-specific deviations
- shrinkage
- controls for late entries and favorites
- uncertainty

## C. Composite model

Define how to combine the above into:

- global trader composite score
- topic composite score
- price-discovery specialist score
- final-outcome specialist score

You must provide:

- an MVP fast heuristic layer
- a more rigorous later empirical Bayes / hierarchical version

The docs explicitly provide the formal direction here. Use it.

# 9. Market-level signal models

Design two live scoring functions.

## A. Repricing signal score

A function like:

`RepricingScore =
a1 * weighted_price_relative_skill

- a2 * topic_matched_skill
- a3 * convergence_strength
- a4 * independence_quality
- a5 * early_entry_signal
- a6 * conviction_proxy
- a7 * neglect_bonus
- a8 * already_repriced_penalty
- a9 * illiquidity_penalty
- a10 * copier_contamination_penalty
- a11 * market_maker_flow_penalty`

## B. Final-resolution signal score

A function like:

`FinalResolutionScore =
b1 * weighted_final_resolution_skill

- b2 * topic_matched_final_skill
- b3 * net_directional_exposure
- b4 * specialist_convergence
- b5 * residual_probability_gap
- b6 * late_entry_penalty
- b7 * crowding_penalty
- b8 * market_maker_flow_penalty
- b9 * low_confidence_penalty`

## C. Combined current opportunity score

A final function that combines both while preserving explainability.

Define:

- all terms
- thresholds
- vetoes
- stale-signal gates
- low-liquidity vetoes
- too-close-to-resolution vetoes
- MM/copier contamination vetoes

# 10. Explicit final-resolution prediction design

This is mandatory.

Design a model that tests:

- baseline: final resolution predicted by current market price alone
- augmented: final resolution predicted by current market price + trader signal features

The docs explicitly propose this test.
You must build it into the architecture.

Define:

- target labels
- feature freeze time
- train/validation/test split
- rolling windows
- evaluation metrics
- calibration checks
- statistical significance tests

# 11. Signal explanation layer

For every live candidate market, the system must explain:

- why it was surfaced
- whether the edge is repricing, final-resolution, or both
- which traders mattered
- whether they are whales or non-whales
- whether they are topic specialists
- what penalties applied

- whether the trade still looks copyable
- what invalidates the thesis

This must be generated deterministically from the signal components.

## 12. Backtesting engine

Design a Polymarket-specific backtest framework.

It must support:

- signal freeze at decision time
- lag-aware execution simulation
- spread/slippage
- size caps
- time-to-resolution effects
- correlated markets
- rolling windows
- train/test separation
- model ablations

It must test both:

- repricing objective
- final-resolution objective

Strategy families to test:

- global-score following
- topic-specialist following
- convergence-only
- whale-only
- non-whale-only
- price-relative-only
- final-resolution-only
- dual-signal composite

The docs are explicit that any claim here must survive out-of-sample and baseline comparisons. Build for falsification, not self-confirmation.

## 13. Minimal API design

Define a thin internal API.

At minimum:

- `/markets/signals`
- `/markets/:id/explanation`
- `/markets/:id/repricing-score`
- `/markets/:id/final-resolution-score`
- `/traders/top`
- `/traders/:wallet`
- `/topics/top`
- `/backtests/runs`
- `/health`

Define:

- response schema
- caching
- internal auth assumptions

## 14. Dashboard contract

Define the dashboard data contract.

The UI must support:

- live ranked opportunities
- repricing vs final-resolution labels
- trader leaderboard
- trader drilldown
- topic leaderboard
- convergence monitor
- market drilldown
- alert history
- paper-trade log

You do not need to fully design the frontend visuals, but you must define the data contract.

## 15. Alerting and workflow

Define MVP alerts such as:

- new convergence event
- new high final-resolution residual event
- market moved beyond copyability threshold
- trader classification changed
- market-maker contamination spike

- signal expired
- paper trade entered/exited

Define:

- operator workflow
- review queue
- audit logs
- paper mode default

## 16. Testing strategy

Define a real test plan.

Include:

- unit tests for feature math
- ingestion tests
- dedupe/idempotency tests
- schema validation tests
- classification tests
- backtest reproducibility tests
- API contract tests
- golden fixtures from Polymarket payloads

## 17. Deployment plan

Provide:

- local development
- staging
- production
- workers / cron
- secrets
- logging
- observability
- data retention
- cost-awareness

## 18. MVP phases

Break implementation into phases.

For each phase list:

- deliverables
- dependencies
- success criteria

Suggested structure:

- Phase 1: ingestion + canonical storage
- Phase 2: wallet tape + crude classification
- Phase 3: price-relative scoring
- Phase 4: final-resolution scoring
- Phase 5: live ranking + explanations
- Phase 6: backtesting + stress tests
- Phase 7: optional paper trading / execution

# 19. FIRST FILES TO GENERATE

At the end, produce a section titled exactly:

`FIRST FILES TO GENERATE`

List the first 20–30 real files Claude Code should create.

For each file include:

- filepath
- purpose
- why it comes early
- key classes/functions

Examples:

- `services/ingest/src/streams/rtds_listener.ts`
- `services/ingest/src/pollers/trades_poller.py`
- `packages/schemas/src/trade.ts`
- `packages/quant/src/features/clv.py`
- `packages/quant/src/features/final_resolution.py`
- `packages/quant/src/classifiers/market_maker.py`
- `packages/quant/src/classifiers/copier.py`
- `packages/quant/src/scoring/trader_price_relative.py`
- `packages/quant/src/scoring/trader_final_resolution.py`
- `packages/quant/src/signals/repricing_score.py`

- `packages/quant/src/signals/final_resolution_score.py`
- `packages/quant/src/backtest/replay_engine.py`
- `services/api/src/routes/markets.py`

## 20. Strict output style

Your output must be:

- implementation-first
- file-tree-heavy
- schema-heavy
- explicit
- opinionated
- modular
- concrete

Do not write a generic essay.
Do not give startup advice.
Do not give vague architecture prose.

Produce:

- exact repo layout
- exact schemas
- exact service boundaries
- exact feature definitions
- exact formulas
- exact API contracts
- exact file generation order

## 21. Final deliverables

End with exactly these sections:

1. `Recommended MVP stack`
2. `Fastest path to first usable signal`
3. `Fastest path to first final-resolution test`
4. `Irreducible custom alpha layer`
5. `Top technical risks`
6. `First code generation batch`

---

# Additional design guidance

Use these defaults unless you have a clearly better alternative:

- Treat `proxyWallet` as the trader identity in MVP.
- Treat wallet clustering as out of scope for first version.
- Treat market-maker filtering as first-class.
- Treat copier detection as first-class.
- Build both repricing and final-resolution models from the start, but allow the repricing model to be simpler.
- Keep execution disabled by default.
- Build explainability from day one.
- Separate research tables from serving tables.
- Prefer official Polymarket repos for plumbing.
- Prefer custom code for alpha logic.

---

# One final hard requirement

Whenever there is a tradeoff between:

- adding more ingestion/execution plumbing
- or implementing the alpha logic

prioritize the alpha logic.

The value of this system is not connectivity.
The value is the **skill-filtered, topic-aware, MM-filtered, copier-filtered, copyability-aware ranking engine** for:

- repricing
- final resolution
- or both

---

Useful source docs this prompt is based on:
GitHub Repo Triage for a Polymarket Skilled-Trader and Undervalued-Bet System, GitHub Repositories for Building a Polymarket Skilled-Trader and Undervalued-Bet Detection System, Find Existing Open-Source and Analytics Infrastructure for a Polymarket Smart-Money System, Find Currently Undervalued Polymarket Bets by Tracking High-Skill Traders and Topic Specialists, and Polymarket Trader-Level Behavior as a Statistically Defensible Alpha Signal.

A strong next step is to turn this into a second Claude Code prompt that forces generation of the **first actual code batch** rather than the architecture.

Here is the second Claude Code prompt, optimized to make it generate the **first actual code batch**, not just the architecture.

---

# Claude Code Prompt

You are a principal engineer, quantitative researcher, and production systems architect.

You have already designed the architecture for a Polymarket trader-intelligence / alpha-detection system.

Now do the next step:

**generate the first actual code batch for the MVP**.

This is not an architecture exercise.
This is a code-generation exercise.

Your task is to produce the **initial working implementation** of a signal-only MVP for a system that:

1. ingests Polymarket market/trade/position data
2. normalizes it into a canonical internal schema
3. computes first-pass trader features
4. classifies traders heuristically
5. computes:
    - price-relative / repricing signals
    - final-resolution signals
6. ranks current markets
7. exposes the results via a thin internal API

Execution must remain disabled.
Paper/simulation mode only.

The code must be modular, production-minded, and runnable with minimal editing.

---

# System objective

The code you generate should create the first usable vertical slice of this system:

## In scope for this code batch

- official Polymarket data ingestion wrappers
- canonical schemas
- database models / migrations
- real-time and polling ingestion scaffolding
- first-pass feature computation
- heuristic trader classification
- trader scoring
- market signal scoring
- explanation generation
- thin API layer
- basic config and test scaffolding

## Out of scope for this code batch

Do not fully build:

- wallet clustering
- complex Bayesian inference
- execution engine
- order routing
- advanced dashboard frontend
- full backtester
- sophisticated ML models

You may leave extension hooks for them.

---

# Required implementation philosophy

Use these defaults unless there is a very strong reason not to:

- **Python** for analytics, scoring, classification, API, and most ingestion jobs
- **TypeScript only if clearly necessary** for official stream client integration
- **Postgres** as the core normalized store
- **SQLAlchemy + Alembic**
- **FastAPI**
- **Pydantic**

- **DuckDB or Parquet snapshots optional**
- **Poetry or uv** for Python dependency management
- **Docker Compose** for local orchestration
- **Redis only if absolutely needed**
- **paper-trading / signal-only mode only**

Prefer the simplest stack that gets to a working signal fastest.

---

# Hard functional requirements

The first code batch must support these objects and flows.

## 1. Canonical objects

Implement code and schemas for at least:

### Market domain

- `Event`
- `Market`
- `Outcome`
- `MarketTag`
- `MarketTopic`
- `ResolutionState`

### Trader domain

- `TraderWallet`
- `TraderProfile`
- `TraderClassification`
- `TraderSnapshot`

### Trading domain

- `RawTradeEvent`
- `NormalizedTrade`
- `PositionSnapshot`
- `ClosedPosition`

- `WalletMarketExposure`

### Price domain

- `PriceSnapshot`
- `OrderbookSnapshot`
- `SpreadDepthSnapshot`

### Feature/scoring domain

- `TraderFeatureRow`
- `TraderScoreSnapshot`
- `TopicScoreSnapshot`
- `MarketSignalSnapshot`
- `SignalExplanationRow`

Implement these as real SQLAlchemy models and corresponding Pydantic schemas.

---

## 2. Ingestion

Implement the first working ingestion layer with clean interfaces for:

### A. market metadata sync

Pull and normalize:

- events
- markets
- tags/topics
- outcome definitions
- status/resolution info

### B. trade ingestion

Pull or receive trade events and persist:

- raw payload
- normalized trade rows

### C. position polling

Pull or scaffold:

- positions
- closed positions
- wallet exposure summaries

### D. price snapshots

Persist:

- best bid/ask or midpoint if available
- market price history points if available

If live streaming is harder in the first pass, create:

- a clean stream interface
- a mock/local adapter
- a polling fallback

The important thing is that the code structure supports real streaming later.

---

## 3. Feature computation

Implement first-pass feature modules for:

### A. price-relative / repricing features

- entry vs later midpoint at configurable horizons
- simple CLV-like score
- lateness-adjusted entry score
- notional-weighted CLV
- median CLV
- topic-specific CLV aggregates

### B. final-resolution features

- normalized realized edge
- residual value vs entry implied probability
- simple proper-scoring-style contribution
- topic-specific final-resolution aggregates

### C. classification features

- same-market two-sided trading ratio
- exposure churn
- holding-period proxy
- burstiness proxy
- volume concentration
- topic concentration
- lag-to-leader proxy
- directional purity proxy

All formulas should be explicit in code comments.

---

# 4. Trader classification

Implement a first heuristic classifier that outputs:

- `whale`
- `serious_non_whale`
- `market_maker_style`
- `directional_discretionary`
- `copier_like`
- `noise_or_weak_signal`

This can be rule-based for now.

You must implement:

- a classification config file
- thresholds in config
- deterministic classification logic
- confidence / warning flags
- notes on why each classification was assigned

The classification output must be stored and queryable.

---

# 5. Trader scoring

Implement two first-pass scoring modules.

## A. Price-relative trader score

Should combine:

- CLV-style edge
- lateness penalty
- directional purity adjustment
- ESS/sample threshold
- market-maker penalty
- copier penalty
- topic overrides

### B. Final-resolution trader score

Should combine:

- normalized realized edge
- proper-scoring-style contribution
- late-entry penalty
- favorite/easy-market penalty
- ESS/sample threshold
- market-maker penalty
- copier penalty
- topic overrides

### C. Composite score

Implement a simple weighted composite:

- global composite
- topic composite
- price-discovery specialist score
- final-outcome specialist score

Make all weights configurable.

---

## 6. Market signal scoring

Implement three market-level scoring modules.

### A. `repricing_score`

Should use:

- weighted price-relative trader skill

- topic-matched trader skill
- convergence strength
- independence proxy
- already-repriced penalty
- illiquidity penalty
- contamination penalty

### B. `final_resolution_score`

Should use:

- weighted final-resolution trader skill
- topic-matched final skill
- residual-probability-gap proxy
- specialist convergence
- late-entry penalty
- low-confidence penalty
- contamination penalty

### C. `current_opportunity_score`

Should combine the two while preserving:

- separate component values
- veto flags
- explanation fields

All scores must return structured explanation data, not just a number.

---

## 7. Explanation system

Implement a deterministic explanation generator.

For each surfaced market signal, return:

- market id / slug
- side
- score breakdown
- whether the signal is:
    - repricing
    - final-resolution
    - both

- top contributing traders
- trader classes
- topic relevance
- applied penalties
- vetoes
- copyability notes
- invalidation notes

The system should be explainable without reading the raw code.

---

## 8. API

Implement a thin FastAPI service with at least these routes:

- `GET /health`
- `GET /markets/signals`
- `GET /markets/{market_id}/explanation`
- `GET /markets/{market_id}/repricing-score`
- `GET /markets/{market_id}/final-resolution-score`
- `GET /traders/top`
- `GET /traders/{wallet}`
- `GET /topics/top`

The API must:

- return JSON
- use Pydantic response models
- support pagination where relevant
- have clean dependency injection
- separate service logic from routes

---

## 9. Database and migrations

Implement:

- SQLAlchemy models
- Alembic migration setup
- first migration files
- seed/dev helper script

The migration set should create the minimum viable schema to support:

- ingestion
- features
- scoring
- API reads

---

## 10. Tests

Implement a meaningful initial test suite.

At minimum include:

- schema/model tests
- normalization tests
- feature math tests
- classification tests
- signal-scoring tests
- API route smoke tests

Use fixtures for:

- sample market payload
- sample trade payload
- sample position payload
- sample trader history
- sample signal case

Do not leave tests as placeholders.
Write real test code.

---

# Required repository structure

Generate a real file tree similar to this, refined if needed:

polymarket_alpha/

  pyproject.toml

  README.md

```
.env.example
docker-compose.yml
alembic.ini

apps/
  api/
    main.py
    dependencies.py
    config.py
    routes/
      health.py
      markets.py
      traders.py
      topics.py
    schemas/
      markets.py
      traders.py
      topics.py

packages/
  core/
    config.py
    logging.py
    constants.py
```

```
  time.py

db/
  base.py
  session.py
  models/
    market.py
    trader.py
    trade.py
    position.py
    price.py
    scoring.py

ingestion/
  clients/
    polymarket_http.py
    polymarket_stream.py
  sync/
    markets_sync.py
    trades_sync.py
    positions_sync.py
    prices_sync.py
  normalize/
    markets.py
```

```
    trades.py
    positions.py
    prices.py

features/
  price_relative.py
  final_resolution.py
  behavior.py
  topic_features.py

classification/
  rules.py
  heuristics.py
  labels.py

scoring/
  trader_price_relative.py
  trader_final_resolution.py
  trader_composite.py
  market_repricing.py
  market_final_resolution.py
  market_opportunity.py
  explanation.py
```

```
  services/
    market_service.py
    trader_service.py
    topic_service.py
    signal_service.py

  tasks/
    refresh_markets.py
    refresh_trades.py
    refresh_positions.py
    recompute_features.py
    recompute_scores.py

infra/
  docker/
    api.Dockerfile

scripts/
  seed_dev_data.py
  run_ingest_once.py
  recompute_all.py

tests/
  conftest.py
```

```
  fixtures/
    markets.json
    trades.json
    positions.json
    traders.json
  test_models.py
  test_normalization.py
  test_features.py
  test_classification.py
  test_scoring.py
  test_api.py
```

If you propose a better structure, explain why.

---

# Output requirements

Your response must include three things.

## Part 1. Final file tree

Give the final concrete repository tree.

## Part 2. Full file contents

Generate the actual contents of the first batch of files.

You must fully implement at least these files:

- `pyproject.toml`
- `.env.example`

- `docker-compose.yml`
- `README.md`
- `packages/core/config.py`
- `packages/db/base.py`
- `packages/db/session.py`
- `packages/db/models/market.py`
- `packages/db/models/trader.py`
- `packages/db/models/trade.py`
- `packages/db/models/position.py`
- `packages/db/models/price.py`
- `packages/db/models/scoring.py`
- `packages/ingestion/clients/polymarket_http.py`
- `packages/ingestion/normalize/markets.py`
- `packages/ingestion/normalize/trades.py`
- `packages/features/price_relative.py`
- `packages/features/final_resolution.py`
- `packages/features/behavior.py`
- `packages/classification/rules.py`
- `packages/scoring/trader_price_relative.py`
- `packages/scoring/trader_final_resolution.py`
- `packages/scoring/trader_composite.py`
- `packages/scoring/market_repricing.py`
- `packages/scoring/market_final_resolution.py`
- `packages/scoring/market_opportunity.py`
- `packages/scoring/explanation.py`
- `packages/services/signal_service.py`
- `apps/api/main.py`
- `apps/api/routes/health.py`
- `apps/api/routes/markets.py`
- `apps/api/routes/traders.py`
- `apps/api/routes/topics.py`
- `tests/conftest.py`
- `tests/test_features.py`
- `tests/test_classification.py`
- `tests/test_scoring.py`
- `tests/test_api.py`

You may generate additional files if needed.

## Part 3. Run instructions

Provide exact commands for:

- install
- run Postgres
- run migrations
- seed test data
- run ingest once
- recompute features/scores
- start API
- run tests

---

# Code quality requirements

All code must be:

- runnable
- typed where reasonable
- modular
- reasonably documented
- free of obvious placeholders
- consistent in naming
- explicit in assumptions
- easy to extend

Do not produce pseudo-code when real code can be written.

If a function depends on unavailable production credentials or live endpoints:

- still implement the function
- make it configurable
- provide a fallback/mock path
- do not leave it blank

---

# Scoring and modeling requirements

Use explicit formulas.
Keep MVP formulas simple but structurally correct.

Examples:

# Price-relative trader score

A simple form may be:

`score =
w1 * shrunk_mean_clv

- w2 * median_clv
- w3 * lateness_penalty
- w4 * mm_penalty
- w5 * copier_penalty
- w6 * directional_purity_bonus`

# Final-resolution trader score

A simple form may be:

`score =
v1 * shrunk_realized_edge

- v2 * proper_scoring_contribution
- v3 * late_entry_penalty
- v4 * easy_market_penalty
- v5 * mm_penalty
- v6 * copier_penalty`

# Market repricing score

A simple form may be:

`score =
a1 * weighted_trader_skill

- a2 * topic_match_bonus
- a3 * convergence_strength
- a4 * already_repriced_penalty
- a5 * contamination_penalty`

## Market final-resolution score

A simple form may be:

`score =
b1 * weighted_final_skill

- b2 * specialist_alignment
- b3 * residual_gap_proxy
- b4 * late_signal_penalty
- b5 * contamination_penalty`

Make weights configurable in code.

---

# Hard implementation rules

1. Do not overengineer streaming on first pass.
   A well-structured polling+adapter design is acceptable.
2. Do not overengineer Bayesian modeling on first pass.
   A clean heuristic/shrunk-score pipeline is acceptable.
3. Do not skip explanation generation.
   Explanation is mandatory.
4. Do not skip contamination handling.
   Market-maker/copier penalties must exist from the start.
5. Do not skip final-resolution support.
   Even if crude at first, it must be present.
6. Do not build execution.
7. Do not hand-wave persistence.
   Use real DB models.
8. Do not return only snippets.
   Return full file contents.

---

# Final instruction

Produce the best possible **first working code batch** for this system.

Prioritize:

- correctness of structure
- fast path to usable signals
- explicit alpha logic
- explainability
- maintainability

Do not default back into architecture prose.

Generate real code.

---

If you want, I can also turn this into a third prompt that is even more aggressive: one that tells Claude Code to generate the code in **multiple passes**, with pass 1 = repo skeleton + DB + schemas, pass 2 = ingestion, pass 3 = features/classification, pass 4 = scoring/API/tests.

Here is the multi-pass Claude Code prompt.

It is tuned to the main conclusions from the docs: official Polymarket repos are good for plumbing, but the alpha layer is still custom; the system should test both repricing and final-resolution edge; raw win rate is structurally confounded; topic specialists and filtered non-whales may matter more than headline whales; and the backtest must include visibility lag, slippage, and “already repriced” penalties.

---

# Claude Code Prompt

You are a principal engineer, quantitative researcher, and production systems architect.

Your task is to build a **Polymarket trader-intelligence / alpha-detection MVP** in **multiple controlled passes**.

This is not a one-shot code dump.
This is a staged implementation process.

The goal is to reduce brittleness, improve correctness, and force clean interfaces between:

- ingestion
- normalization
- canonical storage
- feature computation
- trader classification
- trader scoring
- live signal generation

- backtesting
- API serving

The system must support **two alpha objectives**:

1. **repricing edge**
    - does observed skilled flow predict near-term price movement?
2. **final-resolution edge**
    - do trader-conditioned features improve prediction of final outcome beyond market price alone?

The docs support building both, while treating repricing as the more plausible first edge and final-resolution as something that must be tested explicitly rather than assumed.

# Hard conceptual rules

Do not frame this system as:

- copy the highest win-rate traders
- copy the biggest PnL wallets
- copy whales blindly

That is structurally wrong.

The system must instead:

- filter market makers
- detect copier-like wallets
- model topic specialization
- estimate price-relative edge
- estimate final-resolution edge
- model copyability under real frictions
- produce explainable signals

The docs are explicit that raw win rate and raw PnL are confounded by late entries, heavy favorites, correlated bets, and market-maker activity.

# Repo reality you must respect

Assume:

- official Polymarket repos are strong for CLOB access, live streaming, CLI/operator workflows, and execution plumbing
- `real-time-data-client` is the fastest route to live wallet-tape style ingestion

- `polymarket-cli` is high leverage for MVP control-plane and prototyping
- `clob-client` and `py-clob-client` are the correct base for exchange connectivity
- the real missing layer is custom alpha logic: shrinkage, topic skill, convergence, copyability, MM filtering, copier filtering, and Polymarket-specific backtesting

The docs say this directly.

## Data reality you must respect

Assume the MVP uses first-party public surfaces such as:

- leaderboard
- trades
- positions
- closed positions
- market metadata
- prices history
- CLOB / orderbook style market data
- RTDS / market stream
- proxyWallet as the trader identity object

Assume:

- wallet clustering is not MVP scope
- proxyWallet is enough for the first pass
- Polymarket tags / Gamma metadata should be the first-pass topic taxonomy
- binary markets should be prioritized first

These are the most practical starting assumptions from the docs.

---

# Required implementation method

You must generate the system in **passes**.

Each pass must:

1. produce a clear file tree delta
2. generate full code for the files in scope
3. state any assumptions
4. include tests for what was added
5. keep the code runnable after that pass

Do not skip ahead.
Do not write placeholders where real code can be written.
Do not dump all files at once.

At the end of each pass, provide:

- file tree after pass
- full file contents for new/changed files
- exact commands to run
- what the system can do after this pass
- what remains incomplete

---

# Required stack defaults

Unless there is a very strong reason otherwise, use:

- Python for analytics, API, classification, scoring, tests, and most batch ingestion
- FastAPI
- SQLAlchemy
- Alembic
- Pydantic
- Postgres
- Poetry or uv
- Docker Compose for local dev
- TypeScript only if clearly needed for official streaming integration
- execution disabled by default
- signal-only / paper mode only

---

# Required pass plan

Use the following pass sequence unless you can justify a clearly better one.

## PASS 1 — repo skeleton, config, DB base, canonical schemas

Goal:
Create the project skeleton and the minimum canonical database model.

Must include:

- project tree
- dependency files
- config
- DB session/base
- initial SQLAlchemy models
- initial Alembic setup
- Pydantic schemas
- basic README
- Docker Compose
- smoke tests for DB models

Canonical domain objects must include at minimum:

## Market domain

- Event
- Market
- Outcome
- MarketTag
- MarketTopic
- ResolutionState

## Trader domain

- TraderWallet
- TraderProfile
- TraderClassification
- TraderSnapshot

## Trading domain

- RawTradeEvent
- NormalizedTrade
- PositionSnapshot
- ClosedPosition
- WalletMarketExposure

## Price domain

- PriceSnapshot
- OrderbookSnapshot
- SpreadDepthSnapshot

### Feature/scoring domain

- TraderFeatureRow
- TraderScoreSnapshot
- TopicScoreSnapshot
- MarketSignalSnapshot
- SignalExplanationRow

PASS 1 output requirements:

- final file tree after pass 1
- full code for every new file
- initial migration files
- model tests
- exact run commands

## PASS 2 — ingestion clients, normalization, first batch sync jobs

Goal:
Make the system capable of ingesting market/trade/price/position data into the canonical schema.

Must include:

- HTTP client wrappers for Polymarket data surfaces
- normalization functions
- metadata sync
- trades sync
- positions sync
- prices sync
- idempotent insert/update logic
- mock/fallback mode for offline testing
- ingestion tests

Important:
If live streaming is too much for this pass, implement:

- a stream adapter interface
- a stub/mock stream client
- a polling fallback

Do not overengineer streaming yet.

PASS 2 output requirements:

- file tree delta
- full code for all new/changed files
- sample fixtures
- normalization tests
- ingest-once script
- exact run commands

# PASS 3 — feature computation and heuristic trader classification

Goal:
Compute first-pass features and classify wallets.

Must implement features for:

## Price-relative features

- entry vs later midpoint at configurable horizons
- CLV-like metrics
- lateness adjustment
- odds-difficulty adjustment
- notional-weighted and median variants

## Final-resolution features

- normalized realized value
- residual value vs entry implied probability
- simple Brier/log-style contributions
- topic-level aggregates

## Behavioral features

- both-side same-market ratio
- exposure churn
- holding-period proxy
- burstiness
- directional purity
- lag-to-leader proxy
- topic concentration
- market breadth vs depth

Must implement first heuristic classes:

- whale
- serious_non_whale
- market_maker_style
- directional_discretionary
- copier_like
- noise_or_weak_signal

Use config-driven thresholds.
Store explanation notes with classifications.

The docs explicitly call out MM filtering, copier detection, topic specialization, and ESS/shrinkage logic as core needs.

PASS 3 output requirements:

- file tree delta
- full code
- feature tests
- classification tests
- recompute scripts
- exact run commands

# PASS 4 — trader scoring and market signal scoring

Goal:
Turn features into usable trader scores and market opportunity scores.

Must implement:

## Trader score families

1. price-relative trader score
2. final-resolution trader score
3. composite trader score
4. topic-specific score variants

## Market signal families

1. repricing score
2. final-resolution score
3. combined opportunity score

Each score must preserve structured explanation output.

Scoring design must explicitly include:

- ESS or sample threshold effects
- late-entry penalty
- easy-market / favorite penalty
- market-maker penalty
- copier penalty
- topic-match bonus
- convergence strength
- already-repriced penalty
- illiquidity penalty
- contamination penalties

The docs are explicit that the minimal meaningful version should be convergence-first and CLV-aware, with filtered traders and conservative friction assumptions.

PASS 4 output requirements:

- file tree delta
- full code
- scoring tests
- example signal cases
- exact run commands

# PASS 5 — API layer and explanation layer

Goal:
Expose the system via a thin FastAPI service.

Must include:

- GET /health
- GET /markets/signals
- GET /markets/{market_id}/explanation
- GET /markets/{market_id}/repricing-score
- GET /markets/{market_id}/final-resolution-score
- GET /traders/top
- GET /traders/{wallet}
- GET /topics/top

Must include:

- response schemas
- pagination where relevant
- service layer abstraction
- explanation generator
- API tests

PASS 5 output requirements:

- file tree delta
- full code
- API tests
- exact run commands

# PASS 6 — backtesting and validation harness

Goal:
Add a minimal but real backtest harness.

Must support:

- signal freeze at decision time
- lag-aware entry
- spread/slippage assumptions
- hold-to-resolution mode
- optional fixed-horizon exit for repricing
- rolling windows
- baseline comparisons

Must test at least:

- price-only baseline
- global trader score
- topic-specialist following
- convergence-only
- whale-only
- non-whale-only
- final-resolution-only
- dual-signal composite

The docs say this backtest layer is a major custom gap and must model visibility lag, spread/depth, and already-repriced penalties.

PASS 6 output requirements:

- file tree delta
- full code
- backtest tests
- example run commands
- example result schema

## PASS 7 — optional live stream integration and operator workflow polish

Goal:
Add the first real-time ingestion path and basic operator workflow.

Must include:

- stream client integration using official live client if justified
- signal refresh loop
- alerting hooks
- audit logging
- paper-trade event log

This pass is optional if earlier passes already produce a usable research system.

---

# Required mathematical framing

You must encode the system using explicit formulas.

At minimum include trade-level notions like:

- entry price
- later price at horizon h
- time to resolution
- odds difficulty weight
- notional weight

Use definitions structurally equivalent to:

- CLV-like edge
- realized normalized value
- Brier/log-style forecast contribution
- lateness downweighting
- topic partial pooling later, but not necessarily in MVP code

The docs explicitly define this style of framing and recommend partial pooling later rather than naive win-rate heuristics.

---

# Required output format for each pass

When I ask for a pass, you must respond with exactly these sections:

1. `Pass objective`
2. `Design decisions`
3. `File tree after this pass`
4. `Full file contents`
5. `How to run`
6. `What works now`
7. `What is still missing`

Do not omit any section.

---

# Hard coding rules

1. Keep the system runnable after every pass.
2. Use real code, not pseudocode.
3. Keep functions typed where reasonable.
4. Use comments to make formulas explicit.
5. Put thresholds and weights in config, not hardcoded deep in functions.
6. Preserve explainability at every layer.
7. Default execution to disabled.
8. Prefer simple deterministic heuristics first, richer inference later.
9. Do not skip tests.
10. Do not collapse repricing and final-resolution into one metric.

---

# Initial repo layout target

Use a structure similar to:

polymarket_alpha/

  pyproject.toml

  README.md

```
.env.example
docker-compose.yml
alembic.ini

apps/
  api/
    main.py
    dependencies.py
    config.py
    routes/
      health.py
      markets.py
      traders.py
      topics.py
    schemas/
      markets.py
      traders.py
      topics.py

packages/
  core/
    config.py
    logging.py
    constants.py
```

```
  time.py

db/
  base.py
  session.py
  models/
    market.py
    trader.py
    trade.py
    position.py
    price.py
    scoring.py

ingestion/
  clients/
    polymarket_http.py
    polymarket_stream.py
  sync/
    markets_sync.py
    trades_sync.py
    positions_sync.py
    prices_sync.py
  normalize/
    markets.py
```

```
    trades.py
    positions.py
    prices.py

features/
  price_relative.py
  final_resolution.py
  behavior.py
  topic_features.py

classification/
  rules.py
  heuristics.py
  labels.py

scoring/
  trader_price_relative.py
  trader_final_resolution.py
  trader_composite.py
  market_repricing.py
  market_final_resolution.py
  market_opportunity.py
  explanation.py
```

```
  services/
    market_service.py
    trader_service.py
    topic_service.py
    signal_service.py

  tasks/
    refresh_markets.py
    refresh_trades.py
    refresh_positions.py
    recompute_features.py
    recompute_scores.py

  backtest/
    replay.py
    execution.py
    metrics.py
    strategies.py

infra/
  docker/
    api.Dockerfile

scripts/
```

```
  seed_dev_data.py
  run_ingest_once.py
  recompute_all.py
  run_backtest.py

tests/
  conftest.py
  fixtures/
    markets.json
    trades.json
    positions.json
    prices.json
    traders.json
  test_models.py
  test_normalization.py
  test_features.py
  test_classification.py
  test_scoring.py
  test_api.py
  test_backtest.py
```

Refine it if needed, but preserve this separation.

---

# Very important priority rule

If there is a tradeoff between:

- polishing infrastructure
- and implementing the actual alpha logic

prioritize the alpha logic.

The unique value is not another Polymarket bot.
The unique value is the custom layer that the docs say does not already exist in reusable repo form:

- skill inference
- topic specialization
- convergence scoring
- copyability modeling
- MM filtering
- copier filtering
- Polymarket-specific validation/backtesting

---

# First command

Start with:

```
PASS 1 — repo skeleton, config, DB base, canonical schemas
```

Generate that pass completely.

---

A sharper variant is to start even smaller with only Pass 1A: config + DB + core models. But the full Pass 1 above is still a reasonable first cut given the docs.

Good.

Paste that prompt into Claude Code as-is. It will start with:

```
PASS 1 — repo skeleton, config, DB base, canonical schemas
```

The right way to use it is sequentially:

1. let it complete Pass 1
2. review and fix any issues
3. then tell it: `Continue with PASS 2`
4. repeat through PASS 6 or PASS 7

Do not ask it to jump straight to all passes at once. The staged version is stronger because it keeps the codebase runnable after each layer.

If you want, the next thing to do is have me make a **Pass 1 review checklist** so you can quickly judge whether Claude's first output is actually good.

Here is a Pass 1 review checklist you can use against Claude Code's output.

Pass 1 review checklist

1. Project skeleton is real, not decorative

Check that it produced:

- a coherent repo tree
- `pyproject.toml`
- `README.md`
- `.env.example`
- `docker-compose.yml`
- `alembic.ini`
- app/package folders that match the intended separation

Fail if:

- folders exist but most files are empty
- there is no clear separation between DB, models, API, and shared config
- naming is inconsistent across files

2. Config layer is usable

Check that:

- config is centralized
- environment variables are loaded from one place
- defaults are sensible for local development
- DB URL, app mode, log level, and feature flags are configurable

Fail if:

- secrets or URLs are hardcoded
- config is duplicated across files
- there is no local/dev-safe default behavior

3. DB base/session setup is clean

Check that:

- there is one SQLAlchemy base
- one session factory
- one engine creation path
- session management is explicit and reusable

Fail if:

- multiple bases or engines are created accidentally
- sync/async patterns are mixed incoherently
- session lifecycle is unclear

4. Models actually match the intended domain

Check that the core model set exists and is not collapsed into vague generic tables.

At minimum verify:

- market/event/outcome objects exist
- trader wallet/profile/classification objects exist
- raw and normalized trade objects are separate
- position and closed-position objects exist
- price/orderbook/spread-like objects exist
- scoring/feature snapshot objects exist

Fail if:

- everything is dumped into 2–3 giant tables
- raw and normalized data are mixed
- trader classification is missing
- scoring tables are absent

5. Model fields are structurally correct

Check that each model has:

- primary key
- created/updated timestamps where appropriate
- clear foreign keys
- correct nullable vs non-nullable choices
- unique constraints where identity requires them
- indexes on likely query paths

Examples of expected indexed/query-heavy fields:

- market id / slug
- event id
- trader wallet / proxy wallet
- timestamp fields
- market + side combinations
- score snapshot timestamps

Fail if:

- there are no indexes
- foreign key relationships are vague or absent
- timestamps are missing on append-heavy tables
- identity fields are nullable without reason

6. Append-only vs mutable tables are handled correctly

Check that the design distinguishes:

- append-only event history tables
- latest snapshot tables
- mutable profile/config-style tables

Examples:

- `RawTradeEvent` should generally be append-only
- `NormalizedTrade` usually append-only
- `TraderSnapshot` can be snapshot-style
- `TraderProfile` can be mutable
- score snapshots should usually be append-only by run timestamp

Fail if:

- everything is mutable
- historical state would be overwritten
- no notion of snapshots vs event logs exists

7. Alembic is real, not fake

Check that:

- Alembic is wired to the SQLAlchemy metadata
- there is an initial migration
- the migration creates the intended tables
- the migration can plausibly run without manual surgery

Fail if:

- Alembic files exist but no revision was generated

- migration imports are broken
- model metadata is not discoverable by Alembic

8. Pydantic schemas are distinct from ORM models

Check that:

- API/request/response schemas are separate from SQLAlchemy models
- schema names are consistent with DB models
- schemas are typed cleanly
- nested output shape is sensible

Fail if:

- ORM models are used directly as API models
- response schemas are missing
- fields drift from DB model names without explanation

9. Naming is consistent across the whole codebase

Check consistency for:

- `TraderWallet` vs `WalletTrader`
- `MarketSignalSnapshot` vs `SignalSnapshot`
- `NormalizedTrade` vs `TradeNormalized`
- snake_case in DB fields
- PascalCase for classes
- route/schema names aligned with services/models

Fail if:

- there are multiple names for the same concept
- pluralization is inconsistent
- one file uses “wallet” and another uses “trader” for the same identity layer without distinction

10. The market schema supports Polymarket reality

Check that the market model can represent:

- event
- market
- outcomes
- tags/topics
- resolution state
- binary markets at minimum
- timestamps like end/resolution where available

Fail if:

- outcomes are not modeled separately
- resolution state is missing
- no room exists for topic/tag enrichment

11. The trader schema supports later alpha logic

Check that trader-related tables leave room for:

- classification labels
- quality flags
- topic-specific scoring later
- wallet clustering placeholder
- heuristic notes / reasons
- snapshot history

Fail if:

- there is only a raw wallet string and nothing else
- no place exists for classification confidence or notes
- no room exists for later score/history evolution

12. The scoring schema is good enough for later passes

Check that score tables can support:

- price-relative score
- final-resolution score
- topic-specific score
- component breakdown
- model/version name
- timestamp/run id
- explanation linkage

Fail if:

- only a single opaque numeric score exists
- no versioning or timestamp exists
- no component storage exists

13. The codebase is runnable after Pass 1

Check that you can plausibly do:

- install deps
- start Postgres
- run migrations
- import models
- run tests

Fail if:

- import paths are broken
- package init files are missing where needed
- migrations require manual editing just to boot
- README commands do not match actual files

14. Tests are real and useful

Check that tests cover at least:

- model imports
- table creation/migrations
- basic constraints
- a few relationship expectations
- schema validation

Fail if:

- tests are placeholders
- tests only assert `True`
- no DB fixture exists
- tests do not actually touch the models

15. README is operational

Check that the README tells you:

- what this repo is
- how to install it
- how to run Postgres
- how to run Alembic
- how to run tests
- what Pass 1 currently supports
- what is intentionally not built yet

Fail if:

- README is generic boilerplate
- commands are missing
- setup steps don't match the repo

16. It did not jump ahead and overcomplicate

Pass 1 should not try to fully implement:

- live streaming
- full ingestion
- feature computation

- backtesting
- execution
- heavy ML/Bayesian code

Fail if:

- it starts building later-pass logic in half-broken form
- the core schema layer becomes messy because it tried to do too much

17. It preserved the two-objective structure

Even in Pass 1, the schema should clearly leave room for:

- repricing / price-relative signals
- final-resolution signals

Check for this in:

- score tables
- feature tables
- explanation tables
- naming conventions

Fail if:

- the schema already collapses everything into one generic “alpha score”

18. It preserved explainability

Check that the schema leaves room for:

- signal explanation rows
- classification notes
- component values
- model/version metadata

Fail if:

- the design is already opaque at the storage layer

Fast scoring rubric

Give each section 0, 1, or 2:

- 0 = bad / missing
- 1 = partial
- 2 = solid

Score these 10 buckets:

1. repo structure
2. config
3. DB/session setup
4. core models
5. keys/indexes/constraints
6. migration quality
7. schema separation
8. test quality
9. runnability
10. future-fit for later passes

Interpretation:

- 18–20: good enough to continue to Pass 2
- 14–17: continue only after small fixes
- 10–13: rewrite parts of Pass 1 before continuing
- below 10: do not continue; re-prompt Claude

Fast rejection triggers

Reject Pass 1 immediately if any of these happen:

- no real migration
- no real SQLAlchemy models
- raw and normalized trades are merged
- no scoring/feature snapshot tables
- no trader classification table
- broken imports / unrunnable project
- no meaningful tests

A good follow-up prompt for Claude after review

If Pass 1 is mostly good but flawed, use:

“Revise PASS 1 only. Do not continue to PASS 2. Fix the following issues precisely: [paste issues]. Keep the repo runnable after the revision. Regenerate only changed files plus any required migration/test updates.”

If you want, I can also make a matching Pass 2 review checklist.

Here are review checklists for all passes.

Use the same scoring method for each pass:

- 0 = bad / missing
- 1 = partial

- 2 = solid

And the same interpretation:

- strong: continue
- mixed: fix first
- weak: do not continue

Pass 2 review checklist — ingestion, normalization, first sync jobs

1. HTTP/client layer is real
   Check that there are actual client wrappers for the intended Polymarket surfaces, not just empty placeholders.

Look for:

- base client
- auth/config handling if needed
- timeout/retry logic
- typed response handling
- clear endpoint-specific methods

Fail if:

- raw `requests.get()` calls are scattered everywhere
- endpoints are hardcoded deep inside sync jobs
- no shared client abstraction exists

2. Ingestion layers are separated
   Check that:

- clients fetch
- normalizers transform
- sync jobs persist

Fail if:

- one giant function fetches, transforms, and writes everything
- normalization logic is mixed inside routes or scripts

3. Market metadata sync is credible
   Check that it can ingest:

- events
- markets
- outcomes
- tags/topics
- resolution/status fields

Fail if:

- only market names/slugs are stored
- outcomes are skipped
- metadata updates overwrite history carelessly

4. Trade ingestion preserves raw and normalized forms
   Check that:

- raw payload is stored
- normalized trade rows are stored separately
- duplicate trade handling exists
- timestamps are parsed correctly
- side/outcome/price/size fields are explicit

Fail if:

- raw and normalized trades are merged
- dedupe keys are unclear
- timestamp parsing is fragile

5. Position ingestion is structurally sound
   Check that:

- positions and closed positions are separated if needed
- wallet-market exposure can be rebuilt
- polling cadence assumptions are explicit

Fail if:

- positions are treated as append-only events without snapshot semantics
- wallet exposure is impossible to reconstruct later

6. Price ingestion is usable for later CLV features
   Check that:

- midpoint/bid/ask or equivalent price fields are preserved
- timestamp granularity is sufficient
- market linkage is correct
- history can be queried by time

Fail if:

- only current price is stored
- no historical price series is persisted
- price rows cannot support horizon lookups later

7. Idempotency exists
   Check that sync jobs can be run twice without corrupting data.

Look for:

- upsert logic
- natural keys
- dedupe constraints

- safe rerun behavior

Fail if:

- reruns duplicate rows
- ingestion is not replay-safe

8. Retry and failure behavior are explicit
   Check that:

- retries are bounded
- rate-limit handling exists
- transient failures do not corrupt state
- failed runs log enough to debug

Fail if:

- a single HTTP failure crashes everything
- partial writes leave corrupted records with no logging

9. Mock/fallback mode exists
   Check that:

- offline/test fixtures can drive ingestion
- client layer can be swapped for fixture mode

Fail if:

- tests depend on live endpoints only
- local development is blocked by missing credentials

10. Tests are meaningful
    Check for:

- normalization tests
- client parsing tests
- idempotency tests
- fixture-based ingest tests

Fail if:

- tests only patch everything and assert nothing real
- no sample Polymarket payloads exist

Fast rejection triggers for Pass 2:

- no raw/normalized separation
- no idempotency
- no price history support
- sync jobs are monolithic
- ingestion cannot run from fixtures

Pass 3 review checklist — features and heuristic trader classification

1. Feature groups are properly separated
   Check that there are distinct modules for:

- price-relative features
- final-resolution features
- behavioral/classification features
- topic features

Fail if:

- all features live in one giant file
- repricing and final-resolution logic are mixed beyond recognition

2. Price-relative features are structurally correct
   Check that CLV-like features actually use:

- entry price
- later price at defined horizon
- market/time linkage
- configurable horizons

Fail if:

- CLV is approximated without later price lookup
- horizons are hardcoded and opaque
- sign conventions are inconsistent

3. Final-resolution features are real
   Check that there is actual support for:

- realized normalized value
- residual vs entry implied probability
- simple proper-scoring contribution

Fail if:

- final-resolution support is absent
- they quietly substituted win rate only

4. Behavioral features help classification
   Check for features like:

- same-market two-sided ratio
- churn
- holding proxy
- burstiness
- directional purity
- topic concentration
- lag-to-leader proxy

Fail if:

- classification is based only on wallet size
- there is no attempt to detect MM/copier-style patterns

5. Topic aggregation exists
   Check that:

- features can be grouped by topic
- ESS/sample counts by topic exist
- topic concentration can be measured

Fail if:

- topic specialization is postponed entirely
- no topic feature table or aggregation exists

6. Classifier is config-driven
   Check that:

- thresholds live in config
- labels are deterministic
- notes/reasons are stored

Fail if:

- thresholds are hardcoded inside random functions
- classification output is just one label with no explanation

7. Market-maker heuristics are plausible
   Check for evidence that MM-like behavior is captured, such as:

- two-sided activity
- high churn
- low directional purity
- fast flips
- balanced exposure

Fail if:

- “market maker” means only “large wallet”
- no real MM heuristics exist

8. Copier heuristics are plausible
   Check for:

- lag-to-leader logic
- overlap with other wallets
- repeated delayed imitation patterns
- suspicious correlation structures

Fail if:

- copier detection is omitted
- copier classification is based on hand-wavy labels

9. Effective sample size or minimum sample logic exists
   Check that:
- weak-sample traders are penalized
- not everything gets treated as signal

Fail if:

- tiny-history wallets get strong classifications
- no ESS/min-trade threshold exists

10. Tests actually validate formulas and labels
    Check for:
- deterministic feature tests
- classifier fixture tests
- threshold behavior tests
- edge cases

Fail if:

- tests only verify imports
- classification cannot be regression-tested

Fast rejection triggers for Pass 3:

- no final-resolution features
- no MM/copier heuristics
- no topic logic
- no ESS/min sample gating
- classification not explainable

Pass 4 review checklist — trader scoring and market scoring

1. Two trader score families exist
   Check that there are separate trader scores for:
- price-relative skill
- final-resolution skill

Fail if:

- everything is collapsed into one generic score

2. Composite trader score is explicit
   Check that:
- composite weights are visible
- topic variants exist or are scaffolded
- component storage exists

Fail if:

- composite is opaque
- no way to inspect subcomponents

3. Score penalties are present
   Check for explicit penalties for:

- late entries
- easy/favorite markets
- MM contamination
- copier contamination
- weak sample size

Fail if:

- scores are simple averages of CLV or realized edge
- confounds are ignored

4. Topic matching is actually used
   Check that market scoring uses:

- topic-specific trader skill when available
- not just global skill

Fail if:

- topic-specific tables exist but are ignored in signal ranking

5. Repricing signal exists as its own object
   Check that repricing score uses:

- weighted trader skill
- convergence
- already-repriced penalty
- liquidity/copyability penalties

Fail if:

- repricing score is just “sum of trader scores”

6. Final-resolution signal exists as its own object
   Check that final-resolution score uses:

- final-resolution trader skill
- topic alignment
- residual probability logic or proxy
- confidence penalties

Fail if:

- final-resolution score is missing
- it is secretly the same as repricing score

7. Combined current-opportunity score is transparent
   Check that:

- both component scores remain visible

- vetoes are preserved
- score breakdown is queryable

Fail if:

- combined score hides the repricing/final split
- no explanation rows exist

8. Convergence logic is meaningful
   Check for:

- count of strong traders
- weighted convergence
- independence proxy
- same-side aggregation

Fail if:

- convergence is just number of wallets
- copier contamination is ignored

9. Copyability is modeled
   Check for:

- already-moved penalty
- illiquidity penalty
- stale signal penalty
- near-resolution penalty if relevant

Fail if:

- scores assume frictionless copying

10. Scoring tests are real
    Check for:

- deterministic scoring tests
- contamination penalty tests
- veto tests
- convergence tests
- component breakdown tests

Fail if:

- only happy-path tests exist
- weights or penalties are untested

Fast rejection triggers for Pass 4:

- repricing and final-resolution not separated
- no contamination penalties
- no convergence logic

- no copyability penalties
- no score explainability

Pass 5 review checklist — API and explanation layer

1. Routes are cleanly separated
   Check for:
- health
- markets
- traders
- topics

Fail if:

- all routes live in one file
- service logic is embedded in route functions
2. Service layer exists
   Check that routes call service functions rather than raw DB queries everywhere.

Fail if:

- routes are fat and untestable
3. Response schemas are strong
   Check that:
- Pydantic responses are used
- schemas reflect actual payloads
- pagination types exist where needed

Fail if:

- API returns raw ORM objects
- response shape is inconsistent across routes
4. Markets/signals endpoint is actually useful
   Check that it returns:
- ranked opportunities
- side
- repricing score
- final-resolution score
- combined score
- summary explanation/veto state

Fail if:

- endpoint only returns a score number
- no ranking or filtering support exists

5. Explanation endpoint is substantive
   Check that it returns:
- top contributing traders
- trader classes
- topic relevance
- penalties applied
- vetoes
- whether the signal is repricing/final/both
- invalidation notes

Fail if:

- explanation is a canned string
- explanation ignores score components

6. Traders endpoint is useful
   Check that a trader detail route shows:
- classifications
- score snapshots
- topic score summaries
- reason notes if available

Fail if:

- trader drilldown is too shallow to debug signal quality

7. Topics endpoint is useful
   Check that:
- topic-level rankings or summaries exist
- topic specialization is visible via API

Fail if:

- topic route is decorative only

8. Pagination/filtering is sensible
   Check for:
- limit/offset or cursor
- market/topic filters where useful
- sort options or default ranking

Fail if:

- endpoints dump everything unbounded

9. API tests are real
   Check for:
- route smoke tests
- response contract tests
- fixture-backed endpoint tests

Fail if:

- tests only check 200 status
- no response schema assertions exist

10. Explanation determinism exists
    Check that explanation derives from structured data, not ad hoc prose.

Fail if:

- repeated calls on same state can produce inconsistent explanations

Fast rejection triggers for Pass 5:

- no explanation endpoint
- no service layer
- raw ORM responses
- signals endpoint not actually ranking markets
- no API tests

Pass 6 review checklist — backtesting and validation harness

1. Backtest is time-causal
   Check that signals are frozen at decision time.

Fail if:

- future prices or outcomes leak into signal construction

2. Two objectives are supported
   Check that backtests can evaluate:

- repricing strategies
- final-resolution strategies

Fail if:

- only one is tested
- final-resolution is silently omitted

3. Baselines exist
   Check for baseline comparisons such as:

- price-only
- naive market-following
- random or weak-trader baseline
- maybe convergence-only baseline

Fail if:

- strategy is judged without baselines

4. Lag is modeled
   Check for:
- visibility delay
- entry delay
- already-repriced penalty or realistic fill timing

Fail if:

- fills happen at impossible observed prices
5. Slippage/spread assumptions exist
   Check that:
- execution cost assumptions are configurable
- illiquid markets are penalized or filtered

Fail if:

- backtest assumes zero friction
6. Exit logic is explicit
   Check for:
- fixed horizon exit for repricing
- hold-to-resolution for final-resolution
- clear strategy-specific exits

Fail if:

- exit is ambiguous or inconsistent
7. Strategy families are actually tested
   Look for:
- global score
- topic specialists
- whale-only
- non-whale-only
- convergence-only
- final-only
- composite

Fail if:

- only one favored strategy is backtested
8. Results schema is usable
   Check that results store:
- period
- params
- metrics
- version
- strategy id

- maybe trade log / signal log

Fail if:

- results are just printed to console

9. Validation structure exists
   Check for:

- train/test or rolling windows
- parameter versioning
- out-of-sample discipline

Fail if:

- everything is in-sample only

10. Backtest tests exist
    Check for:

- fixture-based replay tests
- lag/slippage behavior tests
- no-lookahead assertions

Fail if:

- backtest code is untested
- time leakage is not guarded

Fast rejection triggers for Pass 6:

- lookahead leakage
- no friction modeling
- no baselines
- no dual-objective testing
- results not persisted or structured

Pass 7 review checklist — live stream integration and operator workflow polish

1. Streaming integration is bounded and clean
   Check that:

- stream client is isolated
- fallback polling still exists
- message handling is structured

Fail if:

- streaming logic infects the whole codebase
- no fallback exists

2. Stream events map into canonical schema cleanly
   Check that:
- live events go through normalization
- dedupe still works
- timestamps/order are handled carefully

Fail if:

- live events bypass canonical data model
3. Signal refresh loop is safe
   Check that:
- refresh cadence is explicit
- recompute boundaries are clear
- race conditions are reasonably managed

Fail if:

- concurrent updates can corrupt score state easily
4. Alerting is meaningful
   Check for alerts like:
- new convergence
- new final-resolution residual
- signal became uncopyable
- contamination spike
- signal expiry

Fail if:

- alerts are generic “score changed” spam
5. Audit logging exists
   Check that:
- emitted signals are logged
- alert history is stored
- paper-trade events are stored

Fail if:

- operator decisions cannot be reconstructed later
6. Paper mode remains default
   Check that:
- no execution path is turned on by default
- signal logs and paper trades are first-class

Fail if:

- system drifts into half-built execution logic

7. Operator workflow is coherent
   Check that:
- there is a review queue or obvious review flow
- alerts link back to explanations
- signal state transitions are understandable

Fail if:

- operator UX is an afterthought with no state flow

8. Live tests exist where possible
   Check for:
- stream parsing tests
- alert generation tests
- audit log tests

Fail if:

- live layer is untested

9. Observability exists
   Check for:
- logs
- error handling
- basic metrics or at least counters
- health monitoring for stream workers

Fail if:

- stream failures are silent

10. It did not bloat into premature execution infra
    Fail if:
- it spends more time on order routing than live signal quality

Fast rejection triggers for Pass 7:

- live path bypasses normalization
- no audit trail
- no fallback polling
- execution creeping in by default
- no operator review flow

Global cross-pass checklist

Use this after every pass.

1. Repo is still runnable
- install works

- tests run
- imports work
- migrations still apply
2. Naming remains consistent
- models, schemas, services, and routes use same vocabulary
3. Two-objective structure remains intact
- repricing and final-resolution stay separate throughout
4. Explainability remains first-class
- not bolted on later
5. Configuration remains centralized
- thresholds and weights stay in config
6. No hidden leakage
- especially after Pass 4 onward
7. No silent collapse into “copy whales”
- MM/copier/topic logic must remain present
8. Custom alpha logic is prioritized
- not drowned by plumbing

Master scoring rubric across passes

Score each pass on 5 buckets:

1. correctness
2. modularity
3. runnability
4. faithfulness to thesis
5. test quality

0–2 each, max 10 per pass.

Interpretation:

- 9–10: continue
- 7–8: continue after minor fixes
- 5–6: revise before next pass
- below 5: reject and redo pass

Best follow-up prompts for weak passes

If a pass is weak but salvageable:
“Revise PASS X only. Do not continue. Fix these exact issues: [paste list]. Keep the repo runnable after the revision. Regenerate only changed files and any required test/migration updates.”

If a pass is structurally bad:
“Redo PASS X from scratch. Do not continue. Preserve the intended repo layout and two-objective structure. Focus on correctness, runnability, and tests.”

If you want, I can turn this into a one-page scorecard template you can fill out while reviewing each Claude pass.

sure

Here is a compact scorecard template you can reuse for every pass.

Claude Code pass scorecard

Project:
Pass:
Date:
Reviewer:

Overall verdict:

- Continue
- Continue after minor fixes
- Revise before continuing
- Reject and redo

Pass score summary

| Bucket | Score (0–2) | Notes |
|---|---|---|
| Correctness | | |
| Modularity | | |
| Runnability | | |
| Faithfulness to thesis | | |

Test quality

Total / 10

Interpretation:

- 9–10 = continue
- 7–8 = continue after minor fixes
- 5–6 = revise before next pass
- below 5 = reject and redo

Critical fail checks

Mark any that apply:

- No real migration
- Repo not runnable
- Imports broken
- Raw and normalized trades merged
- Repricing and final-resolution collapsed into one score
- No MM filtering logic
- No copier logic
- No explanation support
- No meaningful tests
- Lookahead leakage
- No idempotency
- No copyability/friction modeling where required

If any critical fail is checked:

- Default verdict: Reject or Revise before continuing

Pass-specific checklist

Pass 1 — repo skeleton, config, DB base, canonical schemas

| Check | Score (0–2) | Notes |
|---|---|---|

Repo structure is coherent

Config is centralized and usable

DB base/session setup is clean

Core models match intended domain

Keys/indexes/constraints are sound

Append-only vs mutable tables are handled correctly

Alembic is real and wired correctly

Pydantic schemas are separate from ORM

Naming is consistent

Schema leaves room for both objectives

Explainability fields exist structurally

Tests are meaningful

Pass 1 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 2:

Pass 2 — ingestion, normalization, sync jobs

| Check | Score (0–2) | Notes |
|---|---|---|
| Client layer is real and reusable | | |
| Fetch/normalize/persist layers are separated | | |
| Market metadata sync is credible | | |
| Raw and normalized trade separation is preserved | | |
| Position ingestion is structurally sound | | |
| Price ingestion supports later CLV work | | |
| Idempotency exists | | |
| Retry/failure behavior is explicit | | |
| Mock/fallback mode exists | | |

| | | |
|---|---|---|
| Ingestion tests are meaningful | | |

Pass 2 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 3:

Pass 3 — features and heuristic classification

| Check | Score (0–2) | Notes |
|---|---|---|
| Feature groups are properly separated | | |
| Price-relative features are correct | | |
| Final-resolution features are real | | |
| Behavioral features help classification | | |
| Topic aggregation exists | | |
| Classifier is config-driven | | |
| MM heuristics are plausible | | |
| Copier heuristics are plausible | | |

| | | |
|---|---|---|
| ESS/min-sample logic exists | | |
| Feature/classification tests are meaningful | | |

Pass 3 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 4:

Pass 4 — trader scoring and market scoring

| Check | Score (0–2) | Notes |
|---|---|---|
| Separate trader score families exist | | |
| Composite trader score is explicit | | |
| Confound penalties are present | | |
| Topic matching is actually used | | |
| Repricing score is meaningful | | |
| Final-resolution score is meaningful | | |
| Combined score is transparent | | |

| | | |
|---|---|---|
| Convergence logic is meaningful | | |
| Copyability/friction is modeled | | |
| Scoring tests are meaningful | | |

Pass 4 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 5:

Pass 5 — API and explanation layer

| Check | Score (0–2) | Notes |
|---|---|---|
| Routes are cleanly separated | | |
| Service layer exists | | |
| Response schemas are strong | | |
| Markets/signals endpoint is actually useful | | |
| Explanation endpoint is substantive | | |
| Trader drilldown is useful | | |

| | | |
|---|---|---|
| Topic endpoint is useful | | |
| Pagination/filtering is sensible | | |
| API tests are real | | |
| Explanation is deterministic/structured | | |

Pass 5 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 6:

Pass 6 — backtesting and validation harness

| Check | Score (0–2) | Notes |
|---|---|---|
| Backtest is time-causal | | |
| Both objectives are supported | | |
| Baselines exist | | |
| Lag is modeled | | |
| Slippage/spread assumptions exist | | |

| | | |
|---|---|---|
| Exit logic is explicit | | |
| Multiple strategy families are tested | | |
| Results schema is usable | | |
| Validation structure is out-of-sample aware | | |
| Backtest tests are meaningful | | |

Pass 6 verdict notes:

- Best parts:
- Main issues:
- Required fixes before Pass 7:

Pass 7 — live stream integration and operator workflow

| Check | Score (0–2) | Notes |
|---|---|---|
| Streaming integration is bounded and clean | | |
| Live events map into canonical schema cleanly | | |
| Signal refresh loop is safe | | |
| Alerting is meaningful | | |

Audit logging exists

Paper mode remains default

Operator workflow is coherent

Live tests exist where possible

Observability exists

No premature execution bloat

Pass 7 verdict notes:

- Best parts:
- Main issues:
- Required fixes:

Cross-pass integrity checks

Fill this after each pass.

| Cross-pass check | Yes/No | Notes |
|---|---|---|
| Repo is still runnable | | |
| Naming remains consistent | | |

Config remains centralized

Repricing and final-resolution remain separate

Explainability remains first-class

MM/copier handling remains present

Topic logic remains present

No obvious leakage introduced

Custom alpha logic still prioritized

Decision block

Use this at the end of each review.

Pass:
Total score:
Critical fails:
Decision:

- Continue
- Continue after minor fixes
- Revise pass only
- Redo pass from scratch

Next prompt to Claude:
“Continue with PASS X”
or
“Revise PASS X only. Fix these exact issues: …”

or
“Redo PASS X from scratch. Preserve the two-objective structure and keep the repo runnable.”

Ultra-compact reviewer version

Pass:
Total /10:
Critical fail? Y/N
Decision:
Top 3 problems:
1.
2.
3.

Top 3 good things:
1.
2.
3.

Next action:

- Continue
- Minor fix
- Revise
- Redo

If you want, I can also put this into a clean `.docx` or spreadsheet template.

give background a person would need for all the claude code prompts, passes etc...

I’m assembling a concise technical briefing that explains the project logic, the repo/tool assumptions, the scoring philosophy, and what each Claude pass is supposed to accomplish.

Thought for 13s

Here is the background someone needs before using any of the Claude Code prompts, pass plans, and review checklists.

Overview

The project is not “find the highest win-rate Polymarket wallets and copy them.” The docs argue that this framing is too weak because raw win rate and raw PnL are confounded by late entries, heavy favorites, correlated bets, and market-making activity. The defensible framing is: use Polymarket’s unusually transparent trader- and market-level data to classify participant types, estimate trader-conditioned edge, and test whether those signals improve prediction of future price moves and/or final resolution beyond market price alone.

There are two distinct alpha questions in the project:

1. Does skilled trader flow predict near-term repricing?
2. Does skilled trader flow improve final-resolution prediction beyond current market odds? The docs say the first is more plausible, but the second should still be tested explicitly rather than assumed away.

What Polymarket data makes possible

Polymarket exposes enough public data to build a real MVP. The relevant surfaces include public profile-by-wallet, positions, closed positions, total value, trades, market metadata/tags, price history, and live market/orderbook-style streams. That is why the prompts assume you can build this system without privileged internal access.

The key trader identity object for MVP is the proxy wallet, not a presumed real-world person. The docs explicitly warn that “wallet = person” is not safe because one operator can use multiple wallets, bots can shard activity, and multiple EOAs can fund the same proxy wallet. So the whole early system is wallet-native, with entity resolution deferred.

The first-pass market universe should be binary markets. The docs repeatedly simplify notation around YES/NO binary contracts, implied probability prices, and trade-level scoring on that basis. Multi-outcome markets can come later.

The actual research thesis

The strongest thesis in the docs is not “best wallet overall” but “a subset of traders may show persistent price-relative edge in some domains.” The documents rank topic specialization higher than global leaderboard worship and explicitly say topic specialists and serious non-whales are more promising than high-volume or high-PnL wallets once market makers are filtered out.

The docs also say copying whales blindly is a weak idea. Large wallets can dominate PnL or activity because of market-making, fee rebates, or non-directional microstructure competence rather than because they possess copyable forecasting skill. That is why the prompts keep insisting on whale vs non-whale segmentation, market-maker filtering, and copier detection.

Core quant ideas behind the prompts

There are two main trade-level metrics in the docs.

First is price-relative edge, often framed as CLV-style edge: did the trader buy before the market repriced in that direction? This uses entry price versus a later reference price at a fixed horizon or near resolution. Positive out-of-sample CLV-style edge is the main signal for repricing skill.

Second is realized or final-resolution edge: did the trader buy at odds that were better than the eventual outcome justified? The docs describe normalized realized value and

proper-scoring-rule style contributions such as Brier/log-style logic to separate “being right at a good price” from “being right late.”

The prompts repeatedly penalize three structural confounds:

- lateness: trades near resolution are easier
- odds difficulty: farming heavy favorites inflates hit rate
- correlation: many bets may not be independent evidence
  That is why the docs recommend time-to-resolution downweighting, difficulty weights, effective sample size logic, and eventually empirical Bayes / hierarchical shrinkage instead of naive averages.

Why the project needs classification before ranking

The prompts put so much emphasis on classification because the docs say participant type matters at least as much as headline profitability. You need to distinguish:

- whales
- serious non-whales
- market-maker-style accounts
- directional discretionary traders
- copier-like wallets
- noise wallets
  This is not optional. Without it, you risk mistaking spread-capture or imitation behavior for informational edge.

The docs give practical segmentation logic. Whale status is mostly about notional and market impact. Serious non-whales have smaller size but meaningful sample and consistent edge. Market-makers show both-side activity, low directional purity, churn, and spread-like behavior. Copiers tend to trade shortly after other wallets in the same direction and market with correlated sizing patterns.

Why topic specialization is central

The topic logic in the prompts comes straight from the docs: use Polymarket’s own tags and market organization as the first taxonomy, then refine. The practical hierarchy is category first, then event cluster, then market archetype. That is why the prompts include `MarketTag`, `MarketTopic`, topic score tables, and topic-specific trader scores from the beginning.

The reason is statistical as well as economic. The docs say topic specialization is more coherent than searching for one global “best trader,” and they recommend topic partial pooling so that a trader is not treated as a genius in a tiny niche on weak sample size.

Why copyability and frictions are first-class

Even if a wallet contains real signal, that does not imply the signal is copyable. The docs stress visibility lag, spread, depth, liquidity variation, and the fact that copying itself moves the market. That is why the prompts insist on already-repriced penalties, illiquidity penalties, stale-signal gates, and copyability-aware market scoring.

This is also why the backtest prompts require lag-aware execution, slippage, and baseline comparisons. A backtest that sees a strong wallet buy and then assumes an immediate fill at the same price is not credible.

What the repo/tool findings mean

The repo-related docs are important because they set the engineering boundary correctly. The official Polymarket repos are strong for plumbing: CLOB access, data access, real-time streams, CLI/operator workflows, and execution patterns. But they do not solve the alpha engine. That must be custom.

The highest-leverage official components identified in the docs are:

- CLOB clients (`rs-clob-client`, `clob-client`, `py-clob-client`)
- `polymarket-cli`
- `real-time-data-client`
  These are useful for connectivity, operator workflow, and live wallet-tape style ingestion. They are not trader-skill models.

The docs also say some adjacent repos are useful for scaffolding: `Jon-Becker/prediction-market-analysis` for historical-analysis patterns, `frazane/scoringrules` for proper scoring-rule code, and `tremor` as a monitoring/dashboard shell. But none of these replaces the need for custom trader inference, topic specialization, convergence scoring, or copyability modeling.

What each Claude Code prompt is trying to do

The first set of Claude prompts are architecture prompts. Their job is to force Claude to design a production-minded MVP around the actual research thesis rather than drifting into “build a copy-trading bot.” That is why they emphasize canonical schemas, service boundaries, signal explanation, and the dual-objective structure: repricing and final-resolution. This matches the documents’ conclusion that the real missing layer is the inference engine, not connectivity.

The second set are first-code-batch prompts. They force Claude to stop giving generic architecture and start generating runnable code for the repo skeleton, DB models, ingestion adapters, features, classification, scoring, and API. The reason for that structure is that existing repos solve plumbing, while the missing layer must be encoded in your own database schema and scoring code.

The multi-pass prompt exists because this kind of system is brittle if generated all at once. The docs imply a natural build sequence: canonical storage first, then ingestion, then feature computation and classification, then scoring, then API, then backtesting, then optional live-stream polish. That sequence follows the dependency graph of the actual alpha engine.

What each pass is supposed to accomplish

Pass 1 exists so the whole codebase has a stable ontology: markets, outcomes, wallets, trades, positions, prices, features, scores, and explanations. This matters because the project needs both historical event logs and mutable snapshots, and it needs a place to store both price-relative and final-resolution score families separately. That structure follows directly from the docs' dual-objective framing.

Pass 2 exists to pull real Polymarket data into that schema. The docs support using public profiles, trades, positions, closed positions, value, tags, price history, and RTDS/websocket-style market feeds. The point of this pass is not sophistication; it is idempotent, normalized ingestion that preserves raw and normalized forms.

Pass 3 exists to compute first-pass features and classify wallets. This is where CLV-style metrics, normalized realized value, topic aggregates, directional purity, churn, both-side trading ratio, lag-to-leader, and other behavioral signals enter. It also creates the first heuristic classifier for market-makers, copiers, whales, and serious non-whales. That pass reflects the docs' insistence that classification is part of the alpha thesis, not an auxiliary cleanup step.

Pass 4 exists to convert features into trader scores and market scores. The prompts keep repricing and final-resolution scores separate because the docs treat them as related but distinct hypotheses. This is also the pass where convergence, topic match, contamination penalties, and copyability penalties become explicit parts of ranking live markets.

Pass 5 exists to expose the system in a usable way: ranked signals, explanations, trader drilldowns, topic rankings, and market-specific score breakdowns. The explanation layer matters because the prompts are designed to produce a human-auditable signal engine, not a black box. That is consistent with the docs' focus on identifiable components such as skilled-flow features, topic match, convergence, and penalties.

Pass 6 exists to falsify the idea properly. The docs repeatedly stress that the strategy is only credible if it survives price-only baselines, out-of-sample testing, visibility lag, slippage, and realistic liquidity constraints. That is why the pass plan includes rolling windows, multiple strategy families, and separate repricing/final-resolution evaluation modes.

Pass 7 is optional polish. The docs say live ingestion via the official real-time client is high leverage if timing matters, but it is still downstream of the core alpha logic. So this pass adds streaming, alerts, audit logs, and operator workflow only after the research and scoring layers are already working.

Why the review checklists are strict

The review checklists exist because a model will often drift into the wrong problem:

- too much plumbing, not enough alpha logic
- generic copy-trading architecture
- no raw/normalized separation
- no topic logic
- no market-maker/copier detection
- one opaque alpha score instead of separate repricing and final-resolution logic
  Every checklist item is there to guard against one of those failure modes, all of which are explicitly warned about in the docs.

Minimum mental model before someone starts

A person using these prompts should understand six things.

First, the project is a trader-intelligence and signal-extraction system, not a generic Polymarket bot.

Second, the main unit of identity at MVP stage is the proxy wallet, not a human trader identity.

Third, the main statistical objects are CLV-style edge, realized/final-resolution edge, topic-specific skill, and uncertainty/shrinkage. Raw win rate is not enough.

Fourth, participant classification is core: whales, serious non-whales, market-makers, copiers, and noise wallets must be separated.

Fifth, the live ranking problem is about convergence and copyability as much as it is about trader skill. A strong signal that has already repriced is not useful.

Sixth, official repos help with access and streaming, but the valuable part of the system is the custom alpha layer.

Practical default assumptions for the builder

Start with binary markets only. Use proxy wallet as identity. Use Polymarket tags as the first topic taxonomy. Treat the first live version as signal-only and paper-trading only. Keep execution disabled. Default to simple deterministic heuristics first, then add better shrinkage and partial pooling later. Those defaults are all consistent with the docs' recommended MVP path.

The simplest credible MVP, according to the docs, is:

- ingest leaderboards/trades/positions/prices
- build a first-pass smart-trader set
- filter obvious MM/copier patterns
- compute CLV-style proxies and basic final-resolution metrics
- rank markets by weighted skilled flow minus already-repriced and illiquidity penalties

- expose a watchlist plus explanations
- then add stronger backtests and live streams later.

Biggest things not to get wrong

Do not optimize around copying whales. Do not use raw win rate as the primary signal. Do not treat official repos as solving the alpha problem. Do not ignore market-makers and copiers. Do not collapse repricing and final-resolution into a single uninspected score. Do not trust any backtest that lacks lag, slippage, and price-only baselines. Those are the central warnings across the uploaded docs.

If you want, I can turn this into a shorter “operator briefing” version that is only one page long.